CC/Codex 用户反馈标注说明（26.11领域专项） 副本

一、项目背景

💡 我们希望还原用户在真实使用 Coding Agent 产品过程中的反馈信号，以判断模型是否真正满足了用户需求，并基于这些数据进行后续模型训练与持续优化。为此，我们计划标注一批**真实用户在使用 Coding Agent 过程中的环境、轨迹与高质量用户反馈**数据，共建 Coding Agent 能力。

二、项目周期

🔁
- **11/10 ～ 12/15：**产出**不少于 15,000** 条数据。除爬坡或模型无法提供服务的特殊情况外，每个团队**日产量不应低于 500 条，不应高于 1,500 条。**

🕑 **当天 20:00 前执行任务产生的数据必须当天提交，20:00 后产生的数据允许次日提交。**

所有数据不允许返修，不符合质量要求的数据将直接拒收。

若专家被项目方抽检发现高频不合格数据，这位专家的历史数据全部拒收，不允许继续参与项目。

严格禁止使用 AI 工具分析产物与轨迹！

使用 AI 标注的专家，一经发现，拒收全部历史数据，禁止再次参与项目。

❗ **相比上一期的需求调整：**

- **单轮 Feedback：**本期回归到**收集单轮维度的 pointwise feedback 数据**。要求专家使用 `auto` 模型执行任务，并留存完整环境与产物（初始环境快照、产物快照、运行录屏、轨迹文件）。**每一个 session 仅可进行一轮有效交互，此为一条数据。**
- **聚焦重点领域：**本期出题须围绕指定的重点领域（安全、游戏/图形与多媒体、数据工程）及对应子方向展开。

- **评价维度精简：**除题目与环境/产物基本字段外，仅保留「交付完整性」打分与「Feedback 文本」，不再需要单独对其他维度进行打分和评价。

1. 提交字段说明

🔗 **请各位 TPM 使用下述表头模板，在自己的空间维护正式提交表，并及时将提交表分享给 樊静 。**

可根据内部管理需求添加其它字段，例如截图附件等。

表格

	User Prompt	所属领域	子方向	任务类型
1				

1 条记录

字段	填写说明
题目与环境信息	
User Prompt	本题的完整 prompt 原文，直接粘贴，不要摘要或改写。
所属领域	单选，本题所属的重点领域
子方向	单选，对应所属领域下的细分方向
任务类型	单选，按本题的主要意图归类
任务难度	单选，从人类专家执行的视角判断。**严禁出简单题和中等题**，选项中只保留「困难」和「地狱」。
语言/框架	本题涉及的主要语言与框架，多个用逗号分隔，例如 Go, Gin, PostgreSQL。
Harness	单选，承载模型的客户端（Claude Code 或 Codex CLI）。
Harness 版本	客户端版本号。
操作系统	单选，本次作业所在的操作系统环境（MacOS/Linux 或 Windows）。
环境可复现等级	选填。用于评估这道题后续能否被自动重跑。

初始环境快照	任务开始前的工作区 commit permalink，用 40 位完整 SHA。格式与仓库要求见「快照与仓库要求」。
会话与产物信息（单轮执行）	
SessionID	本次跑的会话 ID。每一个 session 仅可进行一轮有效交互，此为一条数据。
TurnID/PromptID	本轮对话的唯一 ID。Codex CLI 取本轮 `task_started` 事件的 `turn_id`；Claude Code 取本轮 user 消息的 `promptId`。
轨迹文件	本次跑的轨迹文件链接。把 `~/.codex/sessions/` 或 `~/.claude/projects/` 下对应的 jsonl 传到对象存储，这里填可直接下载的链接。
产物快照	本次跑结束后的 commit permalink，用 40 位完整 SHA。模型自己提交了多次就填最后一个，不要 squash；「初始环境快照」必须是它的祖先提交。
运行录屏	本次跑产物的运行录屏链接，要求见下方「录屏要求」。
Feedback 评价	
交付完整性	必填，1–5 分（整数）。考察需求功能点是否完整实现、代码能否运行、有无虚假成功（宣称做完了实际没改或改错）。各档评分标准见「第三步」。
Feedback	必填，反馈文本。结合轨迹过程与最终产物撰写具体、可核验的反馈（包含过程问题与产物问题，指明触发节点、模型实际行为、业务影响及具体文件名、报错信息或未实现的需求点；不需要再拆分指令遵循、任务规划、推理能力、执行能力四个维度）。写作要求见「第三步」及文末「如何写好不满意原因」。
其他	
备注	其他需要说明的问题。

2. 作业流程

第一步：接入模型

> 💡 **我们提供的内部模型已经在网关放行了 1M context window，在下面给出的配置脚本里，也对客户端设置了 1M context window，本期作业请始终保持 max context token = 1000000，请勿自行修改配置，作业前请自查 context 确认无误后再开始。**
>
> **全程使用 auto 模型，不需要自行指定模型，作业期间请勿改动任何配置**（包括 max context token 必须保持 1M）。

配置脚本：

vendor-instruction.zip
11.16KB

每个team的baseurl，key不一样，单独分发，线下培训，自行加中转和并发限制。

第二步：围绕重点领域出题并执行单轮任务

🎯 **专项出题要求：**本期所有题目必须围绕下述重点领域与子方向出题。其中**明确列出的子方向作为重点方向，**请基于对应领域项目提出真实的工程需求，不要为了迎合某个细分方向而刻意构造题目；真实场景中，若任务横跨多个子方向、顺带涉及上下游链路或边界难以精准界定，子方向也可以选择「其他」兜底。

所属领域	子方向	典型任务场景参考
安全	**漏洞检测与攻防**	渗透测试、红队评估、漏洞复现、利用链构建与攻防演练；模糊测试（Fuzzing）、二进制/固件逆向、内存破坏分析、漏洞利用与缓解等。
	身份认证	口令、多因素、OIDC/OAuth/SAML、SSO与身份联邦。 RBAC/ABAC、权限模型、API授权与最小权限。 口令哈希、密钥生命周期、密钥托管、HSM/KMS集成。
	网络与通信安全	密码套件、证书与握手安全。TLS/SSH/IPSec、防火墙、WAF、IDS/IPS、DDoS防护与访问控制。流量分析、恶意流量检测、VPN/隧道与匿名通信。
	系统安全	主机基线、权限、补丁、恶意软件防护与终端检测。 云配置基线、IAM、容器/镜像扫描、运行时防护与可信计算。 分类分级、脱敏、加密、DLP、隐私增强与合规。
	***其他**	软件保护与加壳、数字取证、安全通信与流量检测、终端/容器安全、单点登录（SSO/MFA）、安全风控与策略，或安全真实项目中跨方向混合、顺带涉及上下游链路、边界较模糊的真实任务。
游戏、图形与多媒体	**音视频处理**	音频/视频编解码、封装、转码与码率控制；时间线剪辑、转场、特效、调色、字幕与渲染输出；播放器、流媒体协议、自适应码率、DRM 与低延迟传输等。
	图形渲染	光栅化/光线追踪管线、Shader、光照材质与后处理；三维网格/曲面建模、几何变换、过程式生成与纹理；骨骼/蒙皮动画、粒子、体素与实时场景组织等。
	游戏开发	游戏开发的前后端工程开发；游戏规则、关卡、数值、AI行为、战斗与交互逻辑；游戏引擎(unity)能力的优化和开发。
	图像处理	去噪、超分、锐化、色彩校正、几何校正等传统处理。 图层、抠图、拼接、融合、平面设计与合成。

		有损/无损压缩、编码格式、图像质量评估。
数据工程	**数据采集、爬虫与 RPA**	网页与接口数据抓取、请求调度、解析、去重与结构化落库；浏览器或桌面界面自动化（RPA）、业务流程模拟与表单录入；采集任务编排、增量/全量切换与数据清洗入库等。
	数仓与数据湖	面向数据仓库、数据湖和湖仓的数据组织与建楼，包括0DSDWDDWS/ADS分层、维度/事实表、数据集市、宽表、分区、Schema演进，编写与优化数据处理代码(SQL、Spark、Fink) 抽取-转换-加载、离线作业编排、增量/全量同步与数据清洗 异构数据源连接、Schema演进、序列化与格式转换。
	数据分析与数据应用	描述统计、分布/相关性、假设检验与实验分析。 分类/聚类/关联/预测建模，特征与模型选择，业务解释。 业务报表、指标口径、透视汇总与定时分发。

1. **第一步，围绕上述重点领域出题并记录初始快照。**确认题目属于上述重点领域与子方向，且难度为「困难」或「地狱」。出好题后在已配置就绪的工作区提交一次并 push 到远端，拿到 40 位完整 SHA 的 commit permalink，作为「初始环境快照」。

📹 快照与仓库要求

- **快照格式：**初始环境快照填 commit permalink，格式为 `https://github.com/<org>/<repo>/commit/<40 位完整 SHA>`。使用完整 SHA，不要填短 SHA、分支名或 tag：分支和 tag 会随后续提交移动，事后无法还原当时的状态。仓库需要 push 到评测团队可访问的远端，push 到个人私有仓库等同于没有记录。已提交的快照不要再做 force-push 或 rebase，历史改写会让记录的 SHA 变成不可达对象。
- **凭据不要进仓库：**提交工作区前确认 `.gitignore` 已覆盖 `.env` 以及各类密钥、连接串和 token 文件。需要说明外部依赖时，只写类型与版本，不要写真实连接串、账号或 token。
- **整个会话只提交一次快照：**在首轮开始前提交一次即可，这意味着模型产出的代码变化完全由轨迹里的工具调用记录承载，因此 SessionID 与 TurnID/PromptID 必须准确回填：这两个字段一旦填错或漏填，这一轮的产物就再也无从追溯，会因为无法质检而打回。**根据团队内部管理需要，也可以让大家每一轮都提交一下产物，目前不强制，可以写一个自动提交的hook来提效。**

2. **第二步，使用 `auto` 模型执行单轮任务并留存产物。**全程使用 `auto` 模型跑，**只跑首轮（每一个 session 仅可进行一轮有效交互）**，模型给出回答后本轮即结束，不要追问、不要纠错。跑完后依次留存：

- **产物快照：**提交工作区产物并 push 到远端，拿到 40 位完整 SHA 的 commit permalink（若模型自己提交了多次则填最后一个，不要 squash；「初始环境快照」必须是它的祖先提交）；
- **运行录屏：**录制一段产物实际运行的视频并上传至对象存储；
- **轨迹与会话 ID：**记下本次会话的 `SessionID` 与 `TurnID/PromptID`，导出 `~/.codex/sessions/` 或 `~/.claude/projects/` 下对应的轨迹 `.jsonl` 文件并上传至对象存储。

📹 **录屏要求：**录屏的目的是证明产物真实性，以及专家确实对产物有所衡量。所以录的是**产物的实际运行**，要求如下：**从干净状态启动，完整展示真实输出；无法运行的也必须录，把报错过程录出来**；不要剪掉失败或不好看的片段；时长控制在 90 秒以内，对于文件较大的情况（>10MB），可以传到云存储后在表中填可直接播放或下载的链接。

⚠️ **任务难度**的分级如下表。需要专家根据repo、技术栈、题目**从人类执行的视角**判断**任务难度。**

任务难度	
简单	任务意图明确，主要沿已有模式做机械修改或局部实现；即使改多个文件，只要模式高度一致、核心判断很少，也算简单。 **请注意：首轮任务中，严禁出简单题和中等题。**但是后续轮次中，可能因为模型产物质量不佳而产生较为简单的bugfix，这部分数据是允许提交的。
中等	需要理解局部到跨模块的调用/数据流，做一些实现决策；有若干边界条件或约束，但不需要重新设计架构。
困难	需要整合多个模块/系统约束，做关键设计取舍，处理复杂状态、兼容性、权限、并发、性能或异常链路。
地狱	人执行起来需要极高的决策复杂度，通常需求很开放或约束很隐蔽，需要极其深度项目理解、架构级判断、多步复杂的调试验证，或正确性依赖大量边界场景。即使 SOTA 模型也很难成功交付。

第三步：交付完整性打分与撰写整体 Feedback

🈲 **禁止照抄下面的分数描述，必须结合轨迹中的实证来说明打分原因。**

维度	描述	5分 (完美/超预期)	4分 (优秀/小瑕疵)	3分 (及格/勉强可用)	2分 (不及格/需重构)	1分 (完全不可用/严重事故)

交付完整性 (Delivery)	**任务成败：**需求功能点是否完整实现？代码能不能跑？有无虚假成功（宣称做完了实际没改或改错）？	**一次性完美跑通**。不仅实现了所有明示需求，还主动补全了隐性需求（如边界处理），**零**虚假成功。	**基本完美**。达成所有主要需求，代码可运行，但有极少量的细节遗漏，**无**虚假成功。	**勉强达标**。核心功能可跑，但有明显 Bug 需人工微调；或存在**轻微**的虚假成功（如某非核心文件未保存）。	**未达成主要需求**。代码因逻辑或依赖问题无法运行；或存在**较大比例**的虚假成功（如宣称改了 5 个文件，实际只改了 2 个）。	**完全失败**。代码严重编译报错完全不可运行；答非所问；或出现**极其恶劣**的虚假成功（满口答应但一行代码没写）。

📝 **Feedback 文本撰写要求：**

- **过程与产物都要分析：**虽然不再拆分为多个打分字段，但 Feedback 文本中仍需综合审视模型的**执行过程与最终产物**，说清“过程哪里好/差、产物哪里好/差”。
- **高分/满分同样要写具体实证：**若交付评为 4–5 分，同样须说明已逐项核对的需求点、实际运行验证结果及过程表现，不接受“完美完成”“效果很好”“基本可用”等无法核验的一句话评价。

第四步：提交数据

💬 **对于每一轮对话（一个 prompt-response pair），请单独提交一条数据。**

如果某一轮次由于工程故障/网络波动，而不具备反馈价值，可以不提交这一轮，除此之外，应将会话中的所有有效轮次进行反馈和提交。

提交完整个任务内的轮次后，应打开新的任务窗口和项目路径（如需），再进行下一道题。

3. 不应该考虑的因素

- **推理的时长：**可能受模型部署影响。不要把推理时长考虑进去，对于效率的衡量可以看轮次和篇幅。
- **模型戛然而止（无报错）：**可能受到部署和harness适配的影响。遇到了尽量重新roll，如果实在roll不出来一直无故中断（非api报错）就换题目。
- **网络工程错误（有报错）：**不考虑网络波动、请求失败这类工程故障，遇到就重新 roll，持续报错就联系我。
- **禁止使用 AI 工具分析轨迹与产物、或代写 Feedback。**

三、质检点

1. **整体质量维度**
   - 数据总量统计
   - 整体合格率
   - 数据多样性（是否覆盖了各场景，是否出现了雷同的题目）
2. **标注合规性**
   - 轮次合规（是否超10轮、窗口拆分是否正确）
   - 字段完整（是否有字段缺失）
   - 题目分类准确性（任务类型、技术栈、难度）
   - 打分与描述一致性（是否在原因中提到产物有问题却标注了任务完成，或反之）
3. **描述撰写质量**
   - 语句通顺，表达清晰完整
   - 是否出现了极其明显的过程及产物问题却偷懒没有写出
   - 是否真正看过了过程与产物
   - 是否定位到问题具体环节
   - 是否指出了具体行为/问题
   - 是否存在AI生成痕迹

四、不被允许出的雷同题

> ⚠️ 在大量质检中我们还发现了一些已经被出烂的题目和项目（各种经典小游戏、todo、CRUD模板），见下方整理，这些题目我们都会重点查重并拒收。

🕹️ 游戏 / 图形

- 贪吃蛇， 打砖块， 俄罗斯方块，坦克大战经典小游戏及其变种
- 粒子·物理模拟（星系/烟花/落沙/布料）
- 塔防 / 2D 解谜 / 潜行 / 平台跳跃
- 喂食小动物
- 记忆翻牌 / 连连看

🏢 平台 / 业务系统

- 电商购物车/订单
- 权限 RBAC 后台 · 仓库库存
- 投票问卷 · 考勤 OA · 图书借阅
- 博客 CMS · 医院挂号/问诊
- 外卖点餐 · CRM · IM 私信
- 拍卖 · 停车场 · 工单客服

- 五子棋等棋类· 2048 · 贪吃蛇
- 俄罗斯方块 / 扫雷 / 打地鼠

🖥️ **本地 / 桌面工具**

- 命令行工具 CLI
- 代码片段管理器
- 批量重命名 · 截图标注
- 文件管理/同步 · 书签 · 密码管理器
- 商品+Excel 导入 · 积分商城
- XXX 预约系统

📊 **数据可视化 / 前端页面**

- 报表统计 · Streamlit · CSV 看板
- 记账 · 健康健身 · 菜谱
- 天气 · 番茄钟 · 习惯打卡
- 音乐播放器 · 旅行日记 · 观影记录

五、其他常见问题

🛠️ 任务定义与轮次逻辑

Q：数据提交是以“轮次”为单位吗？“一轮”的具体定义是什么？

A：是的。每一个单轮对话（Single-turn）即为一条独立的数据。

- **一轮 = 一次交互**（用户提问 + 模型回答）。
- 即使是一个复杂的任务进行了多次对话，也需要拆分成多条数据提交。

Q：任务类型是根据当前轮次确定的，还是整个对话确定的？

A：根据当前轮次确定。每一轮的标注都要基于该轮的实际内容。

- *示例：* 第一轮是“代码生成”，第二轮如果是修复第一轮的错误，则第二轮的任务类型应变更为“Bug修复”。

🔄 截断与“继续”处理

Q：对于用户输入了“继续”的轮次，数据应该如何提交？

A：对于出现了模型自己中断，后一轮需要人为输入“继续”的情况，两轮的人评其实是对**同一个提示词**的任务在做打分和评价，此时 **Prompt，turnID/PromptID 都提交前一轮的**，但打分和评价基于"继续"后的轮次（可能不止一次继续）来综合评价任务的完成情况，并且在**【其他问题】**中说明中断的情况。

Q：对于“继续”后的轮次，评价标准是什么？

A：评价标准依然是**上一轮用户希望模型完成的任务**。

⚖️ 评分与验收标准

Q：数据验收是针对整个多轮对话会话，还是单轮数据？

A：针对单轮进行验收，每一个单轮对话是一条数据。

Q：如果模型前几轮翻车，但最后一轮修正了，这几轮该怎么评分？

A：

1. **前几轮：** 通常应该会出现一些低分。
2. **最后一轮：** 针对这一轮的表现来评估，结合专家个人用户体验。
    - 如果这一轮依然是反复自纠、磕磕绊绊才完成，体感可能较差。
    - 如果是突然开智回到正轨、高效完成交付，即使前面轮次有错，最后一轮也可能出现高分甚至满分的情况。

Q：是否可以接受多模态Prompt?

A：我们可以接收多模态数据，如果是多模态，除了prompt之外还需要增加一个字段"prompt附件"上传prompt中的附件，提交上会多一步的时间，不可以因此作为原因而提高AHT，请内部衡量。

🎯 题目多样性与难度要求

Q：对任务的难度上限和下限有要求吗?

A：要求如下：

- **上限：** 无量化要求，**欢迎高难度挑战。**
- **下限：** 有严格限制。**禁止**提交过于简单的题目。

Q：同一个repo可以出多少题目?

A：不限制，但是禁止雷同的repo+雷同的题目（包括第一期的数据）。

Q：什么样的题目是“过于简单的题目”？

A：同时满足下述两种及以上特征的题目，是过于简单的题目，拒收：

维度	"过于简单"的特征
修改范围	单文件局部修改、纯函数级实现（典型如 LeetCode 题、独立工具函数）
上下文依赖	不需要任何 Repo 上下文、纯文本 Prompt 就能完成
交互轮次	大多数模型 1 轮就能完美交付，不需要看报错、不需要规划
技术广度	单一文件、单一语言、不涉及前后端联调或跨服务

*附录：任务类型新定义

任务类型（一级标签）	涵盖场景	口径说明
功能开发	0-1开发、1-n开发	从零搭建并实现全新功能、模块或项目；或在已有功能基础上扩展能力、补充场景或增加变体，并保持与现有设计兼容。
Bugfix	环境/配置/依赖修复、语法/编译修复、业务逻辑修复、运行时异常/稳定性修复、数据正确性/数据修复、其他类	解决环境配置/依赖构建问题、语法类型编译错误、业务逻辑偏差、崩溃/超时/内存泄漏等运行期异常，以及数据计算/读写/迁移错误。
代码重构	函数级重构、文件级重构、仓库级重构	在函数、单文件或跨模块/仓库范围内改善代码结构、职责划分、公共能力或依赖边界，保持外部行为兼容。
功能优化	性能优化、扩展性优化、可观测性提升、安全性加固、逻辑健壮性优化	降低耗时/内存/IO/渲染成本，改善插件化扩展能力，补充日志/指标/追踪，强化权限加密与攻击面收敛，提升边界容错与状态一致性。
依赖升级与迁移	升级库/框架/语言版本、API迁移、跨框架/跨语言移植、存储迁移、平台/云迁移	升级依赖/SDK/语言版本并适配兼容性，迁移旧接口/协议，跨框架或跨语言移植实现，以及存储引擎/Schema 或运行平台/云环境迁移。
质量保障	代码静态分析、单测生成与执行、自动化测试、性能测试、兼容性测试、网络与接口测试、安全与合规测试、稳定性/压力测试、测试报告生成	编写或执行单元测试、端到端/集成自动化测试、性能基准压测、兼容性/接口/安全合规测试、长稳压力测试及测试报告汇总。
项目维护	技术文档维护、代码注释、代码审查、版本控制	维护工程设计/接口文档、补充关键逻辑注释、进行 CodeReview 审查缺陷风险、处理 Git 分支/合并/冲突/回滚等版本管理任务。
技术讨论	仓库代码QA（repo QA）、开发计划建议、技术方案建议、通用知识QA	基于仓库上下文回答实现逻辑与调用链问题，或针对目标功能给出拆解步骤、架构选型、接口设计与实现取舍建议。
问题排查与诊断	报错原因分析、运行异常诊断、安全问题定位、异常排查、环境问题诊断	根据错误日志、堆栈或复现步骤定位根因、触发条件与影响范围（含运行异常、安全漏洞/权限风险、跨环境差异排查）。
部署运维	环境配置、依赖管理、打包构建、发布/部署	配置开发/测试/运行环境与脚本，维护依赖版本锁定与冲突处理，配置编译打包构建脚本，处理部署流程与上线验

		证。
泛Agent需求	内容创作、项目运行、skill相关、数据分析、本地文件处理、其他	报告/文档生成、协助项目启动运行调试、Agent/Codex skill 开发调试、数据/日志统计清洗提炼、本地文件批量处理等。

*存档：如何写好不满意原因

💡 在提交不满意原因时，请先在脑海中问自己三个问题，想清楚：

1. **“模型是从哪一步开始降智的？”**
2. **“模型的具体什么操作让我觉得很弱智？”**
3. **“换做是一个人类高级工程师，遇到这种情况会怎么做？”**

“不满意原因”的硬性要求（质检会看）

代码块

```
1．过程与产物都要分析，要分别说清"过程哪里差、产物哪里差"。

2．定位到具体环节和步骤：指明问题发生在哪个步骤、哪个工具调用、哪次文件改动。

3．用证据，不用形容词：引用报错信息、出错的文件名/函数、漏掉的具体需求；禁止只写"效果不好/不行/一般"等笼统的表面症状。

4．只能写由模型本身能力造成的问题，不能写环境/网络波动导致的问题，例如模型请求失败。
```

写作方法参考

💡 一条**合格**的原因必须同时具备三要素：

1. **触发节点（When）**：在哪个阶段或哪个动作出了问题？
2. **实际行为（What）**：模型做错了什么？
5. **业务影响（Impact）**：这个错误导致了什么后果？

一条好的原因，除此之外还需要能够指出模型错误行为的根因和正确做法。

一条数据可能存在很多个问题点，每一点都应按三要素展开说明，写作格式无需完全一致（不作为质检点），但是上述硬性要求都需要踩到（作为质检点）。

可以参考的文本格式（禁AI生成）

过程问题：
在【第几步/哪个环节】，模型【做了什么具体行为】→ 导致【什么后果】（→ 【根因和正确做法】）

产物问题：
产物在【哪个文件/功能】存在【什么具体问题】，表现为【客观证据，如报错/缺失需求】，无法满足【哪条需求】。

产物问题可能是由过程问题导致的，这时可以是更为完整闭环的表述：
在【第几步/哪个环节】，模型【做了什么具体行为】→ 导致产物在【哪个文件/功能】存在【什么具体问题】，表现为【客观证据，如报错/缺失需求】，无法满足 【哪条需求】，（→ 【根因和正确做法】）。

典型的句子示例：

❌ 不合格（一定会被拒收）	✅ 合格
"没完成任务，不满意"	"prompt 要求支持分页，产物只实现了列表查询，完全没动分页参数，产物不完整。"
"代码有 bug"	"产物 utils/date.py 的 parse() 未处理时区，输入带 Z 的 ISO 串会抛 ValueError。"
"(一段意味不明的报错信息)"	"在第 3 步用 edit_file 改 auth.py 时引入了未定义变量 token，且未运行就宣称完成，导致代码报错。"
"过程比较乱"	"规划混乱：模型在第 2、4、6 步三次重复读取同一个 config.py，且第 5 步改完没运行就宣称完成，存在验证缺失"
"效果还行但不满意"	"任务虽完成，但模型臆造了 lodash.deepMerge（实际不存在该方法，应为 merge），仅因该分支未被执行才没报错，属于是出现了幻觉"
"思考超限需要人工继续"	"效率差：一个单文件改动用了 18 步，其中 9 步是反复试错式的 grep，且触发一次思考超限需人工'继续'"
“文件没改成功，一直在报错。”	“在修改 utils.py 时，模型没有先使用 Read 工具读取文件，就直接尝试用 SearchReplace，导致匹配不到对应的代码而报错，模型应该在修改任何已有文件前，必须先 Read 获取最新的上下文才是合理的。”
“写的代码根本不符合我的需求。”	“任务是‘优化当前页面的加载速度’。模型花了很多步骤去修改后端的数据库查询（实际行为），但我明确在提示词里说过‘只允许修改前端缓存逻

	辑’（违反的约束）。期望模型在规划阶段（Plan）严格遵守作用域限制，不要越界修改（期望行为）。”
“越改问题越多。”	“在终端返回了 npm install 的版本冲突错误后，模型连续 3 次尝试了完全相同的安装命令。期望它在遇到此类错误时，能主动读取 package.json 检查依赖版本，而不是盲目重试。”
“需求实现得不完整。”	“对结果不满意的原因如下：虽然三个显式条件全部答对，D_i公式、GAE masking、chunk hidden state 都正确，测试写得也规范。但题目要求的 warm-start（soft-switch teacher + behavior cloning）、前6步probe 协议、eval.py 均未实现，deliverable 残缺。此外存在一个未被自己测试捕获的真实 bug:PPO update 时 chunk 横跨 episode 边界不重置 hidden state，与rollout 时行为不一致。 对过程不满意的原因：模型把"注意事项"当 checklist 来对号入座，只处理被显式点名的三条，对需要自己发现的系统性缺失（warm-start 整个训练阶段缺失）没有意识到。是“做对了被考的点"而不是“做完了这个项目”。”

找问题的方向

> 💡 **我们这里虽然给出了写作/找问题的方法作为参考，但是绝对禁止把这些内容喂给 AI 让 AI 来找问题、写原因！这一行为一经发现，我们不会给专家/供应商任何一次多余的机会，立刻中止合作，请不要挑战我们对AI生成内容的敏感度🙏。**

在读模型过程和产物时，我们需要关心的问题包括**但不限于**：

1. **过程类常见问题**

类别	典型表现
指令遵循	模型是否严格遵守了用户在 Prompt 中明确提出的所有约束条件（如：特定库的使用、代码风格、禁止修改的文件等）？是否存在忽略明确约束、漏做部分需求、做了没要求的额外改动、擅自改技术栈等行为？
任务规划	模型是否能合理拆解任务？是否有清晰的阶段性状态追踪？遇到歧义时是否主动求问？是否有阶段性总结或反馈？是否无规划/规划错误、步骤顺序混乱、目标漂移、来回反复？
工具使用	模型是否能够调用合理的工具？是否出现了参数错误，改文件前不先读、同一失败调用反复重试、不会用搜索等问题？
幻觉	模型是否出现幻觉？是否编造看似合理、实则不存在的 API/参数/配置项，或臆造了 repo 里的文件/函数，声称做了实际没做的操作？

验证缺失	模型是否没有执行合理的测试验证就说完成？是否对报错视而不见、不检查输出？
纠错能力	模型陷入死循环、重复同一种无效修复、或者遇到困难就轻易放弃？
交互沟通	该问不问/不该问乱问、进度汇报误导、缺乏阶段性总结等

2. 产物类常见问题

类别	典型表现
正确性	编译/运行失败、逻辑错误、边界 case 挂、引入回归
完整性	漏需求、半成品、留 stub/TODO
代码质量	可读性差、无错误处理、结构差、违反 repo 规范
过度工程	无意义的高复杂度、多余文件
偷工减料/造假	硬编码、假数据、删/改测试来hack、占位实现
破坏性	删改无关代码、破坏既有功能
安全	引入漏洞
UI/美观度	难看、与设计不符、布局错乱